Réalisation d'un simulateur de CPU

SADI Dalia, 21313803.
HUSSE Léna, 21212867.
LU2IN006.
Sorbonne Université.
Le 29 avril 2025.

Sommaire :

Introduction **2**
Description des structures et du code **2**
- Structures manipulées 2
- Organisation du code et la description des algorithmes créés 5

Un schéma illustrant ce fonctionnement **15**
Jeu d'essai **16**
Analyse des performances **34**
Conclusion **35**
Annexes **36**
Références **42**

Introduction:

Un processeur, ou unité centrale de traitement (CPU), est le cœur d'un système informatique, chargé d'exécuter les instructions d'un programme en coordonnant l'accès à la mémoire, en effectuant des calculs et en contrôlant le flux d'exécution. Il manipule des registres (AX, BX, SP) et la mémoire via des stratégies d'adressage.
Dans ce contexte, ce projet implémente un simulateur d'assembleur simplifié en C, capable de lire un fichier texte (fichier_assembleur.txt), d'allouer des segments mémoire (DS, SS, CS) et d'exécuter un ensemble d'instructions (MOV, ADD, PUSH, POP).
Ce rapport présente la reformulation du sujet, les structures utilisées, l'organisation du code, les algorithmes adaptés, les tests réalisés et une analyse des performances.

Description des structures et du code:

- ## Structures manipulées:

Le simulateur repose sur sept structures principales, qui modélisent la gestion de la mémoire et le fonctionnement de la CPU :

HashEntry:

```
typedef struct hashentry{
        char* key;
        void* value;
}HashEntry;
```

Un entrée dans une table de hachage HashMap pour associer des clés (key) à des valeurs (value).
Les clés sont des chaînes de caractères et les valeurs sont des pointeurs génériques.
Par exemple:
Dans l'exercice 2, on l'utilise pour associer le nom du segment à un pointeur vers ce segment.
Dans l'exercice 3, on l'utilise pour associer le nom d'une variable à son adresse mémoire, ou une étiquette à sa position dans le tableau code_instructions.
Dans l'exercice 4, on l'utilise pour associer les registres à leurs adresses mémoire.

HashMap:

```
typedef struct hashmap{
	int size;
	HashEntry* table;
}HashMap;
```

Elle représente une table de hachage, tel que size est la taille de cette table et table est une HashEntry pour faire les associations.
Utilisée pour stocker les segments (DS, SS, CS), les variables (x, y), les étiquettes (loop, start) et les registres (AX, CX, SP).

Segment:

```
typedef struct segment{
	int start; //Position de début (adresse) du segment dans la mémoire
	int size; //Taille du segment en unites de memoire
	struct segment* next; //Pointeur vers le segment suivant dans la liste chainee
}Segment;
```

Elle représente un bloc mémoire pour gérer les segments DS, SS et CS. Tel que start pour indiquer la position du début de ce segment dans la mémoire, size est le nombre de cases mémoire qu'il occupe et le pointeur next pour se pointer vers le segment suivant.

MemoryHandler:

```
typedef struct memoryHandler{
```

```
    void** memory; //Tableau de pointeurs vers la mémoire allouée
    int total_size; //Taille totale de la mémoire gérée
    Segment* free_list; //Liste chainee des segments de mémoire libres
    HashMap* allocated; //Table de hachage (nom, segment)
}MemoryHandler;
```

Pour représenter l'état global de la mémoire. Tel que memory est un tableau de pointeurs génériques qui représente la mémoire, total_size est la taille de la mémoire (donc la longueur du tableau memory), free_list est une liste chaînée pour y stocker les segments non alloués dans la mémoire et allocated est une table de hachage pour gérer les segment alloués dans la mémoire.
Utilisée dans l'exercice 2, pour gérer les segments, ceux qui sont alloués dans la mémoire (memory et allocated) et ceux qui n'y sont pas (free_list). Et dans l'exercice 4, pour gérer le CPU.

Instruction:

```
typedef struct {
  char * mnemonic ; // Instruction mnemonic (ou nom de variable pour
.DATA)
  char * operand1 ; // Premier opérande (ou type pour .DATA)
  char * operand2 ; // Second opérande (ou initialisation pour .DATA)
} Instruction ;
```

Représente une instruction comme son nom l'indique. Tel que mnemonic est une chaîne de caractère pour dire quelle instruction (MOV, ADD, JMP, …), operand1 est une chaîne de caractère pour représenter le premier opérande de l'instruction qui est souvent un registre ou une chaîne vide si mnemonic est HALT et operand2 qui est une chaîne de caractère pour représenter le deuxième opérande de l'instruction qui peut être une variable, une chaîne vide (par exemple si mnemonic est JMP), une valeur, … .

ParseResult:

```
typedef struct {
  Instruction ** data_instructions ; // Tableau d'instructions .DATA
  int data_count ; // Nombre d'instructions .DATA
  Instruction ** code_instructions ; // Tableau d'instructions .CODE
  int code_count ; // Nombre d'instructions .CODE
```

```
  HashMap * labels ; // labels -> indices dans code_instructions
  HashMap * memory_locations ; // noms de variables -> adresse mémoire
} ParserResult ;
```

C'est le résultat du parsing d'un fichier assembleur. Tel que data_instructions est un tableau de pointeurs sur les instructions stockées dans le segment de données DS, data_count est le nombres de ces instructions, code_instructions est un tableau de pointeurs sur les instructions stockées dans le segment de code CS, code_count est le nombres de ces instructions, labels et memory_locations sont des tables de hachage.

CPU:

```
typedef struct{
  MemoryHandler* memory_handler;
  HashMap* context;
}CPU;
```

Utilisée pour simuler un processeur. Tel que memory_handler un gestionnaire de mémoire et context pour gérer les registres.

CPU:

```
typedef struct{
  MemoryHandler* memory_handler;
  HashMap* context;
  HashMap* constant_pool;//Table de hachage pour stocker les valeurs
immédiates
}CPU;
```

C'est comme la version précédente en lui rajoutant une table de hachage constant_pool pour associer à une variable son adresse mémoire.

- **Organisation du code et la description des algorithmes créés:**

Le code est modulaire, réparti en plusieurs fichiers .c et .h, tel que pour chaque exercice un fichier .h, un fichier .c et un fichier de test main.c, On a choisi de le faire ainsi pour bien gérer les différentes structures et de bien gérer et comprendre les erreurs surtout celles de segmentations :

exo1.h, exo1.c, main1.c :

Le but de l'exercice 1 est d'implémenter une table de hachage générique. Dans le fichier exo1.h, on définit les structures HashEntry et HashMap et les deux constantes TABLE_SIZE pour la taille de la mémoire et TOMBSTONE pour la mettre dans la table de hachage comme value quand on supprime la valeur associée à une clé dans la table.
Dans le fichier exo1.c, on implémente les fonctions utilisées:

simple_hash : La fonction de hachage qu'on a choisie, elle fait la somme du code ASCII des caractères qui composent la clé et pour ne pas dépasser la taille de la table elle retourne le reste de la division de la somme sur la taille de la table.

hashmap_create : Alloue dynamiquement une HashMap et une HashEntry, et initialise toutes les clés et leurs valeurs dans la table de HashMap à NULL. Elle retourne cette HashMap allouée.

hashmap_insert : Insère une clé-valeur dans la table de hachage. Elle calcule l'indice via simple_hash, puis utilise le sondage linéaire (incrémentation modulo TABLE_SIZE) pour trouver une case libre ou une clé existante. Si la clé existe deja dans la table elle met sa valeur à jour en libérant l'ancienne valeur, si la clé n'existe pas ou sa valeur est TOMBSTONE (une case supprimée), cette clé est dupliquée et insérée.

hashmap_get : Récupérer la valeur associée à une clé donnée en argument en suivant le même sondage linéaire. Si la clé existe et que sa valeur n'est pas TOMBSTONE on retourne cette valeur, sinon retourner NULL.

hashmap_remove : Supprime une clé en la recherchant via le même sondage linéaire. Une fois trouvée, la clé est libérée (free), et la case est marquée comme TOMBSTONE pour préserver la continuité du sondage. Retourne 0 (succès) ou 1 (échec).

hashmap_destroy : Libère la mémoire allouée pour la table. Elle parcourt la table, libère les clés dynamiques (key), puis libère le tableau (table) et la structure HashMap. Les valeurs ne sont pas libérées, leur gestion étant laissée aux fonctions appelantes.

exo2.h, exo2.c, main2.c:

Le but de l'exercice 2 est d'implémenter un gestionnaire de mémoire qui la divise en segments alloués (allocated) ou libres (free_list).
L'exercice 2 utilise les fonctions de l'exercice 1.

Dans le fichier exo2.h, on définit les structures Segment et MemoryHandler.
Dans le fichier exo2.c, on implémente les fonctions utilisées:

segment_init : Alloue dynamiquement un segment mémoire de taille size donnée en argument et une adresse début qui est par défaut à 0, elle met next à NULL, et elle retourne le pointeur vers ce segment, NULL en cas d'échec.

memory_init : Initialise un gestionnaire de mémoire avec une taille size passée en argument. Elle alloue un tableau memory de pointeurs initialisés à NULL, une liste free_list avec un unique segment couvrant toute la mémoire, et une table de hachage allocated.

find_free_segment : Recherche un segment libre pouvant contenir un segment de taille size à l'adresse start. Elle parcourt free_list, vérifie si un segment satisfait seg->start <= start et seg->start + seg->size >= start + size, et retourne le segment trouvé avec son prédécesseur (prev) ou NULL si aucun segment n'est disponible.

create_segment : Elle utilise find_free_segment pour vérifier si un segment libre est disponible, alloue un segment nommé name à l'adresse de début start avec la taille size., l'ajoute à allocated via hashmap_insert (donc c’est un segment alloué dans la mémoire), met à jour memory, et ajuste free_list (suppression ou redimensionnement du segment libre).

remove_segment : Libère un segment nommé name en le supprimant de allocated via hashmap_remove, met à jour memory en mettant les cases occupées par ce segment à NULL, et réinsère le segment dans free_list. Une fusion avec les segments libres adjacents (précédent et suivant) est effectuée pour réduire la fragmentation (La fusion combine le segment libéré avec les segments libres adjacents (précédent et/ou suivant) dans free_list pour former un segment libre plus grand).
La fusion est effectuée dans deux cas : Fusion avec le segment précédent : Si le segment libre précédent dans free_list touche directement le segment libéré (c'est-à-dire que son adresse de fin est égale à l'adresse de début du segment libéré), ou fusion avec le segment suivant : Si le segment libre suivant touche directement le segment libéré (c'est-à-dire que l'adresse de début du segment suivant est égale à l'adresse de fin du segment libéré).

free_memory_handler : Libère toutes les ressources du gestionnaire. Elle parcourt memory pour libérer les données des segments DS et SS (que les pointeurs int*, car (*Segment) seront libérés dans allocated), libère les

segments de allocated (pointeurs Segment*), détruit la table de hachage via hashmap_destroy, libère free_list, et enfin libère memory et handler.

exo3.h, exo3.c, main3.c:

Le but de l'exercice 3 est d'implémenter un analyseur syntaxique pour un fichier assembleur.
L'exercice 3 utilise les fonctions de l'exercice 1.
Dans le fichier exo3.h, on définit les structures Instruction et ParserResult.
Dans le fichier exo3.c, on implémente les fonctions utilisées :

trim : Supprime les espaces, tabulations et retours à la ligne au début et à la fin d'une chaîne, renvoyant un pointeur vers la chaîne nettoyée.

parse_data_instruction : Analyse d'une ligne de la section .DATA pour créer une structure Instruction. Elle calcule l'adresse de la variable en utilisant une variable statique next_adr_data, insère l'adresse dans memory_locations via hashmap_insert, et incrémente next_adr_data selon le nombre de valeurs séparées par des virgules.

parse_code_instruction : Analyse d'une ligne de la section .CODE pour créer une structure Instruction. Si un label est présent, il est extrait et inséré dans labels avec son indice d'instruction (code_count).

parse : Lit le fichier assembleur ligne par ligne, identifie les sections .DATA et .CODE, et appelle parse_data_instruction ou parse_code_instruction selon la section. Elle stocke les instructions dans les tableaux dynamiques data_instructions et code_instructions de ParserResult, redimensionnés si nécessaire via realloc. Les tables labels et memory_locations sont créées pour stocker les étiquettes et adresses des variables.

free_parser_result : Libère la mémoire allouée pour ParserResult.

exo4.h, exo4.c, main4.c:

Le but de l'exercice 4 est d'implémenter les fonctions de base pour initialiser et gérer un CPU simulé.
L'exercice 4 utilise les fonctions de l'exercice 1, 2 et 3.
Dans le fichier exo4.h, on définit la structure CPU.
Dans le fichier exo4.c, on implémente les fonctions utilisées :

cpu_init : Alloue et initialise une structure CPU. Elle crée un memory_handler via memory_init, une table context via hashmap_create, et alloue dynamiquement quatre registres (AX, BX, CX, DX) initialisés à 0. Ces registres sont insérés dans context via hashmap_insert.

cpu_destroy : Libère la mémoire allouée pour le CPU. Elle libère les registres (AX, BX, CX, DX) récupérés via hashmap_get, détruit la table context via hashmap_destroy, libère les données dynamiques dans memory (uniquement les int*, pas les Segment*), et libère memory_handler via free_memory_handler avant de libérer cpu.

store : Stocke une donnée data dans le segment nommé segment_name à la position relative pos. Elle récupère le segment via hashmap_get, vérifie sa validité et ses limites, et place la donnée à l'adresse absolue seg->start + pos dans handler->memory.

load : Charge une donnée depuis le segment nommé segment_name à la position relative pos. Comme store, elle récupère le segment, vérifie sa validité, et retourne la donnée à l'adresse seg->start + pos ou NULL en cas d'erreur.

allocate_variables : Alloue le segment DS pour stocker les variables de la section .DATA. Elle calcule la taille totale nécessaire en comptant les valeurs (séparées par des virgules), crée DS via create_segment, et stocke chaque valeur dans DS via store après conversion en int avec atoi.

print_data_segment : Affiche le contenu du segment DS. Elle récupère DS via hashmap_get et parcourt ses positions (start à start + size) pour afficher les valeurs entières stockées dans memory.

exo5.h, exo5.c, main5.c:

L'exercice 5 étend la structure CPU de l'exercice 4, son but est d'implémenter des fonctions gérant différents modes d'adressage.
L'exercice 5 utilise les fonctions de l'exercice 1 et 2.
Dans le fichier exo5.h, on définit la structure CPU.
Dans le fichier exo5.c, on implémente les fonctions utilisées:

matches : Vérifie si une chaîne string correspond à une expression régulière pattern.

cpu_init : Comme la fonction de l'exercice 4, en lui ajoutant l'allocation dynamique de la table de hachage constant_pool pour les constantes.

cpu_destroy : Comme la fonction de l'exercice 4, en lui ajoutant la libération de la table de hachage constant_pool via hashmap_destroy.

immediate_addressing : Gère l'adressage immédiat. Elle vérifie si l'opérande est un nombre (via matches avec ^-?[0-9]+$), stocke la valeur

dans constant_pool si elle n'existe pas, et retourne un pointeur vers cette valeur.

register_addressing : Gère l'adressage par registre. Elle vérifie si l'opérande est un registre (^[ABCD]X$) et retourne sa valeur depuis context via hashmap_get s'il y existe, NULL sinon.

memory_direct_addressing : Gère l'adressage direct. Elle vérifie le format (^\[([0-9]+)\]$), extrait l'adresse entre crochets, et charge la valeur depuis DS à cette adresse via load, elle retourne cette valeur.

register_indirect_addressing : Gère l'adressage indirect par registre. Elle vérifie le format (^\[([ABCD]X)\]$), extrait le registre, récupère sa valeur (qu'on considère comme une adresse) depuis context, et charge la valeur depuis DS à cette adresse via load, elle retourne cette valeur.

handle_MOV : Exécute l'instruction MOV en copiant la valeur de src vers dest.

setup_test_environment : Crée un environnement de test en initialisant un CPU de taille 1024, en définissant des valeurs pour les registres (AX=3, BX=6, etc.), et en allouant un segment DS avec des valeurs de test (5, 15, 25, ...)

store : Stocke une donnée dans un segment à la position relative pos, similaire à l'exercice 4.

load : Charge une donnée depuis un segment à la position relative pos, similaire à l'exercice 4.

resolve_addressing : Résout le mode d'adressage d'une opérande en testant successivement immediate_addressing, register_addressing, memory_direct_addressing, et register_indirect_addressing. Elle retourne la valeur correspondante ou NULL si aucun mode n'est valide.

exo6.h, exo6.c, main6.c:

L'exercice 6 complète le simulateur de CPU en implémentant des fonctions pour résoudre les constantes (labels et variables), allouer le segment de code (CS), exécuter des instructions avec gestion des sauts (JMP, JZ, JNZ), et simuler un programme de manière interactive.
L'exercice 6 utilise les fonctions de l'exercice 1, 2 et 3.
Dans le fichier exo6.h, on définit la même structure que l'exercice 5, CPU.
Dans le fichier exo6.c, on implémente les fonctions utilisées:

search_and_replace : Remplace une chaîne str par sa valeur numérique extraite d'une table de hachage values via hashmap_get.

resolve_constants : Pour chaque instruction dans code_instructions, elle remplace les labels dans operand1 si operand2 est vide par leurs indices via search_and_replace avec labels, et les variables dans operand2 si operand1 est un registre par leurs adresses via memory_locations.

cpu_init : Initialise le CPU avec memory_handler, constant_pool, context, et les registres (AX, BX, CX, DX, IP, ZF, SF) à 0. Étend l'exercice 5 en ajoutant IP, ZF, SF.

cpu_destroy : Libère le CPU, y compris les registres, constant_pool, context, et memory_handler, en itérant sur les tables pour libérer les valeurs dynamiques.

allocate_code_segment : Alloue le segment CS dans memory_handler pour stocker les instructions, en utilisant un segment libre de taille suffisante qu'on a cherché dans free_list. Elle copie les instructions passées en argument dans memory et initialise IP à 0 pour qu'il pointe sur la première instruction de CS.

handle_instruction : Exécute une instruction selon son mnémonique.

execute_instruction : Gère les instructions avec 0, 1, ou 2 opérandes. Si l'instruction est avec deux opérandes, alors operand1 doit forcément être un registre et avec la fonction register_addressing on récupère son adresse qui est vue ici comme dest, si operand2 est une valeur numérique (une adresse) elle récupère la donnée stockée à cette adresse via la fonction load, sinon elle utilise la fonction resolve_addressing qui détecte le mode d'adressage et récupère la valeur résultat qui est vue comme src, enfin elle exécute l'instruction via la fonction handle_instruction. Si l'instruction accepte un seul opérande et que cette opérande operand1 est une valeur numérique qui est vue comme dest et src est à NULL, on appelle la fonction handle_instruction. Si l'instruction n'accepte aucun opérande alors appeler handle_instruction avec src et dest à NULL.

fetch_next_instruction : Récupère l'instruction suivante à exécuter qui est à l'adresse (CS.start + IP) dans memory, après l'avoir récupérée elle incrémente IP pour passer à l'instruction qui suit, et vérifie les limites de CS.

run_program : Simule l'exécution interactive en affichant l'état initial de DS et des registres, en récupérant chaque instruction via fetch_next_instruction, et en demandant à l'utilisateur de valider (Entrer) ou quitter (q). Affiche l'état des registres après chaque exécution et l'état final.

On implémente aussi les fonctions utilisées dans l'exercice 4, qui sont : **allocate_variables** et **print_data_segment**.
Ainsi que celles utilisées dans l'exercice 5, qui sont : **store**, **load**, **immediate_addressing**, **register_addressing**, **memory_direct_addressing**, **register_indirect_addressing**, **handle_MOV** et **resolve_addressing**.
On a copié-collé ces fonctions au lieu juste de faire dans exo6.h #include "exo4.h" et #include "exo5.h" pour eviter toute ambiguïté. Car la structure CPU utilisée dans l'exercice 5 et l'exercice 4 n'est pas la même et aussi dans les exercices 4, 5 et 6 on utilise des fonctions qui ont le même nom mais pas le même corps.

exo7.h, exo7.c, main7.c:

L'exercice 7 complète le simulateur de CPU en implémentant des fonctions pour résoudre les constantes (labels et variables), allouer le segment de code (CS), exécuter des instructions avec gestion des sauts (JMP, JZ, JNZ), et simuler un programme de manière interactive.
L'exercice 7 utilise les fonctions de l'exercice 1, 2 et 3.
Dans le fichier exo7.h, on définit la même structure que l'exercice 5, CPU.
Dans le fichier exo7.c, on implémente les fonctions utilisées:

cpu_init : Initialise le CPU avec memory_handler, constant_pool, context, alloue et initialise les registres (AX, BX, CX, DX, IP, ZF, SF, SP, BP) à 0 et elle les insère dans context via hashmap_insert, ensuite elle alloue le segment DS de taille 256 via create_segment, ensuite alloue le segment SS qui suit DS dans la mémoire avec create_segment, enfin elle initialise les registre BP et SP (à l'initialisation la pile est vide).

cpu_destroy : Libère le CPU, y compris les registres, constant_pool, context, et memory_handler, en itérant sur les tables pour libérer les valeurs dynamiques.

push_value : Empile une valeur sur le segment de la pile SS. Elle récupère SP et SS avec hashmap_get, elle vérifie si la pile est pleine avant de faire l'insertion. Elle décrémente SP pour stocker la valeur donnée en argument à cette adresse là (SP il va pointer sur la nouvelle valeur insérée).

pop_value : Dépile une valeur de SS et la stocke dans une destination. Elle récupère SP et SS avec hashmap_get, elle vérifie si la pile est vide avant de dépiler. On copie dans la destination la valeur sur laquelle SP

pointe ensuite on la libère de memory et on incrémente SP pour qu'il pointe sur la valeur suivante dans le segment de la pile.

handle_instruction : Étend la fonction handle_instruction de l'exercice 6 en lui ajoutant l'exécution de PUSH, POP et HALT.

execute_instruction : Étend la fonction execute_instruction de l'exercice 6.

insert_data : Une modification de la fonction allocate_variable de l'exercice 6, pour qu'elle n'alloue pas le segment de données DS vu qu'il est alloué dans cpu_init donc, elle fait juste l'insertion des données dans DS.

On implémente aussi une fonction utilisée dans l'exercice 4, qui est : **print_data_segment**.
Et on implémente aussi des fonctions utilisées dans l'exercice 5, qui sont : **store**, **load**, **immediate_addressing**, **register_addressing**, **memory_direct_addressing**, **register_indirect_addressing**, **handle_MOV** et **resolve_addressing**.
Ainsi que celles utilisées dans l'exercice 6, qui sont : **matches**, **search_and_replace**, **resolve_constants**, **allocate_code_segment**, **fetch_next_instruction** et **run_program**.
On a copié-collé ces fonctions au lieu juste de faire dans exo7.h #include "exo4.h", #include "exo5.h", #include "exo6.h" pour eviter toute ambiguïté. Car la structure CPU utilisée dans l'exercice 5 et l'exercice 4 n'est pas la même et aussi car dans les exercices 4, 5, 6 et 7 on utilise des fonctions qui ont le même nom mais pas le même corps.

exo8.h, exo8.c, main8.c:

L'exercice 8 complète le simulateur de CPU en implémentant un nouveau segment (l'Extra Segment, ES) et en nous demandant de gérer la gestion de l'allocation dynamique dans la mémoire pendant l'exécution du programme, tout en incluant également l'implémentation de fonctions pour résoudre les constantes (labels et variables), allouer le segment de code (CS), exécuter des instructions avec gestion des sauts (JMP, JZ, JNZ), et simuler un programme de manière interactive pour tester son bon fonctionnement.
L'exercice 8 utilise les fonctions de l'exercice 1, 2 et 3.
Dans le fichier exo8.h, on définit la même structure que l'exercice 5, CPU.

Dans le fichier exo8.c, on implémente les fonctions utilisées:

segment_override_addressing : Gère l'adressage avec préfixe de segment explicite [SEGMENT:REGISTRE]. Valide le format avec une expression régulière, extrait le segment et le registre, et retourne la donnée stockée à la position spécifiée.

find_free_address_strategy : Trouve un segment libre selon la stratégie d'allocation (First Fit, Best Fit, Worst Fit).

resolve_addressing : Étendue pour intégrer le nouveau mode d'adressage avec segments.

cpu_init : Initialise le CPU avec memory_handler, constant_pool, et context. Alloue et initialise les registres (AX, BX, CX, DX, IP, ZF, SF, SP, BP) à 0 et les insère dans context via hashmap_insert. Alloue le segment DS de taille 256 via create_segment, puis alloue le segment SS qui suit DS dans la mémoire avec create_segment. Initialise les registres BP et SP (pile vide à l'initialisation) et ajoute le registre ES au context avec une valeur initiale de -1.

cpu_destroy : Libère le CPU, y compris les registres, constant_pool, context, et memory_handler, en itérant sur les tables pour libérer les valeurs dynamiques.

alloc_es_segment : Alloue dynamiquement un segment ES en utilisant la taille dans AX et la stratégie dans BX. Met à jour ES avec l'adresse de base du segment alloué et utilise ZF pour signaler le succès ou l'échec.

free_es_segment : Libère le segment ES et réinitialise ES à -1.

handle_instruction : Étend la fonction de l'exercice 6 en ajoutant l'exécution de PUSH, POP, HALT, ainsi que les nouvelles instructions ALLOC et FREE pour gérer l'allocation et la libération de l'extra segment.

run_program : Simule l'exécution interactive en affichant l'état initial des segments DS et ES ainsi que des registres, incluant ES. Récupère chaque instruction via fetch_next_instruction, demande à l'utilisateur de valider (Entrer) ou quitter (q), et gère l'arrêt via

HALT. Affiche l'état des registres et des segments DS et ES après chaque exécution, ainsi que l'état final.

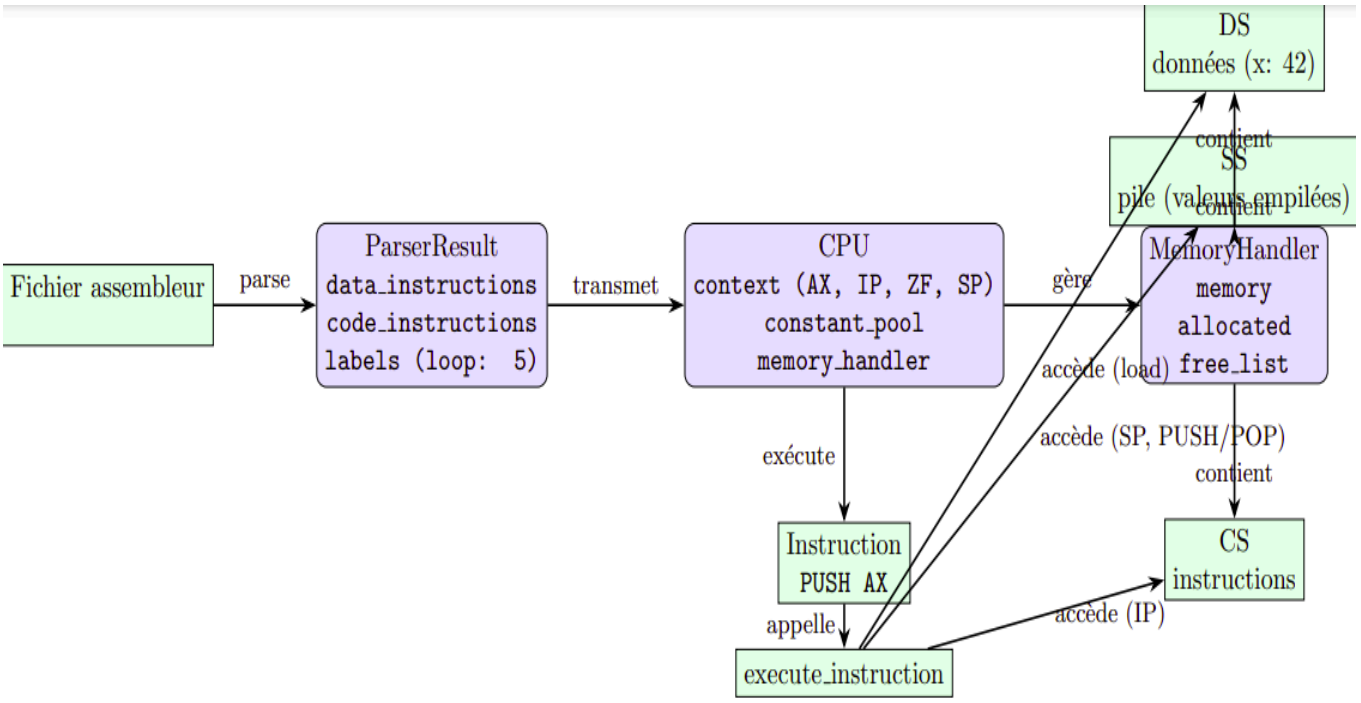
On implémente aussi des fonctions des exercices précédents. Celles de l'exercice 7 que l'on implémente sont :

push_value, **pop_value**, **matches**, **search_and_replace**, **resolve_constants**, **allocate_code_segment**, **fetch_next_instruction**, **insert_data**, **immediate_addressing**, **register_addressing**, **memory_direct_addressing**, **register_indirect_addressing** et **handle_MOV**.

On implémente aussi des fonctions utilisées dans l'exercice 4, qui sont :
print_data_segment, **store** et **load.**

Ces fonctions sont copiées dans exo8.c au lieu juste de faire dans exo8.h #include "exo4.h", #include "exo7.h", #include "exo5.h", pour éviter toute ambiguïté. Car la structure CPU utilisée dans l'exercice 5 et l'exercice 4 n'est pas la même et aussi car dans les exercices 4, 5, 7 et 8 on utilise des fonctions qui ont le même nom mais pas le même corps.

Un schéma illustrant ce fonctionnement:



Jeu d’essai:

Chaque exercice à son jeu de test qui est dans son fichier main.c, on se sert aussi des fichiers assembleur.
Le fichier fichier_assembleur.txt dedans on trouve deux sections .DATA et .CODE chacune suivie de ses instructions :

```
.DATA
x DW 42
arr DB 20,21,22,23
y DB 10
.CODE
start: MOV AX,x
loop: ADD AX,y
JMP loop
```

Le fichier fichier_CODE.txt on y trouve que la section .CODE et ses instructions :

```
.CODE
start: MOV AX ,x
loop: ADD AX ,y
JMP loop
```

Le fichier fichier_DATA.txt on y trouve que la section .DATA et ses instructions :

```
.DATA
x DW 42
arr DB 20,21,22,23
y DB 10
```

Ensuite, le fichier exo7.txt qu’on utilise que dans l’exercice 7, ce fichier contient deux sections .CODE et .DATA avec des instructions de la pile et l’instruction HALT :

```
.DATA
x DW 42
y DB 10
.CODE
start: MOV AX,x
PUSH AX
MOV AX,y
POP BX
HALT
```

Enfin, le fichier exo8.txt qu'on utilise seulement dans l'exercice 8, ce fichier contient les deux sections .DATA et .CODE, ainsi que des instructions similaires à l'exercice 7 tout en rajoutant les mnémoniques ALLOC et FREE pour tester notre allocation dynamique dans ES :

```
.DATA
x DW 3
y DB 10
a DW 50
b DW 3
.CODE
start:
MOV AX, a
MOV BX, b
ALLOC
MOV [ES:AX], x
FREE
HALT
```

main1.c:

L'objectif est de valider les fonctions de l'exercice 1, pour cela on alloue une HashMap avec hashmap_create, on y insère trois paires clé-valeur ("clé1" -> "Valeur1", "clé2" -> "Valeur2", "clé3" -> "Valeur3") avec hashmap_insert et on essaie de récupérer ces valeurs avec hashmap_get, après on supprime une clé "clé2" de la table avec hashmap_remove, enfin on libère la mémoire allouée dynamiquement avec hashmap_destroy.

Sortie attendue:

```
L'insertion de la clé clé1 dans la table est faite
L'insertion de la clé clé2 dans la table est faite
L'insertion de la clé clé3 dans la table est faite
clé1 a comme valeur Valeur1
clé2 a comme valeur Valeur2
clé3 a comme valeur Valeur3
La suppression de la clé clé2 de la table est faite
Après la suppression de clé2 :
Clé non trouvée : clé2
clé2 a comme valeur NULL
exo 1 Libération de la clé clé1 à l'index 108
exo 1 Libération de la clé clé3 à l'index 110
exo 1 HashMap destroyed: 2 keys freed
```

Valgrind:

```
==10878==
==10878== HEAP SUMMARY:
==10878==     in use at exit: 0 bytes in 0 blocks
==10878==   total heap usage: 6 allocs, 6 frees, 3,106 bytes allocated
==10878==
==10878== All heap blocks were freed -- no leaks are possible
==10878==
==10878== For lists of detected and suppressed errors, rerun with: -s
==10878== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

main2.c:

L'objectif est de valider les fonctions de l'exercice 2 en testant l'implémentation du gestionnaire de mémoire.
On alloue un MemoryHandler de size 1000, on y crée avec create_segment deux segments de nom "segment1" et "segment2" et de taille 200 et 300 respectivement, donc ils seront insérés dans allocated, on les affiche, puis on supprime "segment1" avec remove_segment, enfin on libère la mémoire allouée avec free_memory_handler.

Sortie attendue:

```
L'insertion de la clé segment1 dans la table est faite
Segment 'segment1' créé avec succès.
L'insertion de la clé segment2 dans la table est faite
Segment 'segment2' créé avec succès.
Les segments alloués:
Nom du segment: segment1, Taille: 200
Nom du segment: segment2, Taille: 300
La suppression de la clé segment1 de la table est faite
Segment 'segment1' supprimé avec succès.
Clé non trouvée : DS
Clé non trouvée : CS
Clé non trouvée : SS
exo 1 Libération de la clé segment2 à l'index 37
exo 1 HashMap destroyed: 1 keys freed
```

Valgrind:

```
==11206==
==11206== HEAP SUMMARY:
==11206==     in use at exit: 0 bytes in 0 blocks
==11206==   total heap usage: 10 allocs, 10 frees, 11,186 bytes allocated
==11206==
==11206== All heap blocks were freed -- no leaks are possible
==11206==
==11206== For lists of detected and suppressed errors, rerun with: -s
==11206== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

main3.c:

L'objectif est de valider les fonctions de l'exercice 3.
On lui donne le nom d'un fichier assembleur en argument, on alloue un ParserResult en parsant le fichier avec parse, et on affiche tous ses composants, à la fin on libère la mémoire allouée avec free_parser_result.

Sortie attendue:

```
L'insertion de la clé x dans la table est faite
Variable 'x' insérée à l'adresse 0, cpt=1
L'insertion de la clé arr dans la table est faite
Variable 'arr' insérée à l'adresse 1, cpt=4
L'insertion de la clé y dans la table est faite
Variable 'y' insérée à l'adresse 5, cpt=1
L'insertion de la clé start dans la table est faite
Label 'start' inséré à l'indice 0
Instruction parsée: mnemonic='MOV', operand1='AX', operand2='x'
L'insertion de la clé loop dans la table est faite
Label 'loop' inséré à l'indice 1
Instruction parsée: mnemonic='ADD', operand1='AX', operand2='y'
Instruction parsée: mnemonic='JMP', operand1='loop', operand2=''
----- Section .DATA -----
x DW 42
arr DB 20,21,22,23
y DB 10
x a comme valeur 0
arr a comme valeur 1
y a comme valeur 5
data_count a comme valeur 3
```

```
-----Section .CODE -----
MOV AX x
ADD AX y
JMP loop
start a comme valeur 0
loop a comme valeur 1
code_count a comme valeur 3
exo 1 Libération de la clé start à l'index 46
exo 1 Libération de la clé loop à l'index 58
exo 1 HashMap destroyed: 2 keys freed
exo 1 Libération de la clé arr à l'index 69
exo 1 Libération de la clé x à l'index 120
exo 1 Libération de la clé y à l'index 121
exo 1 HashMap destroyed: 3 keys freed
```

Valgrind:

```
==11377== HEAP SUMMARY:
==11377==     in use at exit: 0 bytes in 0 blocks
==11377==   total heap usage: 44 allocs, 44 frees, 10,174 bytes allocated
==11377==
==11377== All heap blocks were freed -- no leaks are possible
==11377==
==11377== For lists of detected and suppressed errors, rerun with: -s
==11377== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

main4.c:

L'objectif est de valider les fonctions de l'exercice 4.
On lui donne le nom d'un fichier assembleur en argument, on alloue un ParserResult en parsant le fichier avec parse, initialiser le CPU de taille 1024 avec cpu_init, allouer le segment DS avec ses variables en utilisant allocate_variables, afficher le contenu de DS avec print_data_segment enfin on libère la mémoire allouée avec cpu_destroy et free_parser_result.

Sortie attendue:

```
L'insertion de la clé x dans la table est faite
Variable 'x' insérée à l'adresse 0, cpt=1
L'insertion de la clé arr dans la table est faite
Variable 'arr' insérée à l'adresse 1, cpt=4
L'insertion de la clé y dans la table est faite
Variable 'y' insérée à l'adresse 5, cpt=1
L'insertion de la clé start dans la table est faite
Label 'start' inséré à l'indice 0
Instruction parsée: mnemonic='MOV', operand1='AX', operand2='x'
L'insertion de la clé loop dans la table est faite
Label 'loop' inséré à l'indice 1
Instruction parsée: mnemonic='ADD', operand1='AX', operand2='y'
Instruction parsée: mnemonic='JMP', operand1='loop', operand2=''
L'insertion de la clé AX dans la table est faite
L'insertion de la clé BX dans la table est faite
L'insertion de la clé CX dans la table est faite
L'insertion de la clé DX dans la table est faite
L'insertion de la clé DS dans la table est faite
Le contenu du segment DS :
42
20
```

```
21
22
23
10
exo 1 Libération de la clé AX à l'index 25
exo 1 Libération de la clé BX à l'index 26
exo 1 Libération de la clé CX à l'index 27
exo 1 Libération de la clé DX à l'index 28
exo 1 HashMap destroyed: 4 keys freed
Clé non trouvée : CS
Clé non trouvée : SS
exo 1 Libération de la clé DS à l'index 23
exo 1 HashMap destroyed: 1 keys freed
exo 1 Libération de la clé start à l'index 46
exo 1 Libération de la clé loop à l'index 58
exo 1 HashMap destroyed: 2 keys freed
exo 1 Libération de la clé arr à l'index 69
exo 1 Libération de la clé x à l'index 120
exo 1 Libération de la clé y à l'index 121
exo 1 HashMap destroyed: 3 keys freed
```

Valgrind:

```
==11973==
==11973== HEAP SUMMARY:
==11973==     in use at exit: 0 bytes in 0 blocks
==11973==   total heap usage: 68 allocs, 68 frees, 22,629 bytes allocated
==11973==
==11973== All heap blocks were freed -- no leaks are possible
==11973==
==11973== For lists of detected and suppressed errors, rerun with: -s
==11973== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

main5.c:

L'objectif est de valider les fonctions de l'exercice 5 pour les modes d'adressage.

On crée un environnement de test avec setup_test_environnement, qui retourne un pointeur vers le CPU, ensuite on utilise register_addressing pour obtenir des pointeurs vers les registres AX, BX, CX. On exécute l'instruction MOV via la fonction handle_MOV, mais, on récupère d'abord

le pointeur vers son deuxième opérande qui est donné en argument de handle_MOV avec différents modes d'adressage, et à chaque exécution de handle_MOV on affiche le contenu du registre AX qui est toujours le registre destinataire (tous les résultat de MOV seront stockés dans AX). Enfin, on teste la fonction resolve_addressing avec différents modes d'adressage et on libère la mémoire allouée.

Sortie attendue:

```
L'insertion de la clé AX dans la table est faite
L'insertion de la clé BX dans la table est faite
L'insertion de la clé CX dans la table est faite
L'insertion de la clé DX dans la table est faite
Clé non trouvée : DS
L'insertion de la clé DS dans la table est faite
Test environment initialized .


Clé non trouvée : 7
L'insertion de la clé 7 dans la table est faite
MOV AX, 7 -> AX = 7
MOV AX, CX -> AX = 100 (CX = 100)
MOV AX, [2] -> AX = 25
Source ou destination invalide
MOV AX, [BX] -> AX = 25 (BX = 6)


Le mode d'adressage de l'opérande 7 est l'adressage immédiat sa valeur est
 :
7
```

```
Le mode d'adressage de l'opérande CX est l'adressage par registre sa valeu
r est :
100
Le mode d'adressage de l'opérande [2] est l'adressage direct sa valeur est
 :
25
Aucun adressage n'est trouvé pour l'opérande [BX]
Clé non trouvée : IP
Clé non trouvée : ZF
Clé non trouvée : SF
exo 1 Libération de la clé AX à l'index 25
exo 1 Libération de la clé BX à l'index 26
exo 1 Libération de la clé CX à l'index 27
exo 1 Libération de la clé DX à l'index 28
exo 1 HashMap destroyed: 4 keys freed
exo 1 Libération de la clé 7 à l'index 55
exo 1 HashMap destroyed: 1 keys freed
Clé non trouvée : CS
Clé non trouvée : SS
exo 1 Libération de la clé DS à l'index 23
exo 1 HashMap destroyed: 1 keys freed
```

Valgrind:

```
==12112==
==12112== HEAP SUMMARY:
==12112==     in use at exit: 0 bytes in 0 blocks
==12112==   total heap usage: 1,400 allocs, 1,400 frees, 148,221 bytes all
ocated
==12112==
==12112== All heap blocks were freed -- no leaks are possible
==12112==
==12112== For lists of detected and suppressed errors, rerun with: -s
==12112== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

main6.c:

L'objectif est de valider les fonctions de l'exercice 6 pour exécuter un programme assembleur de manière interactive.
On alloue un CPU de size 2048 avec cpu_init, on parse le fichier fichier_assembleur.txt avec parse, générant un ParserResult, ensuite, on appelle resolve_constants pour associer les labels (start, loop) et variables (x, y) à leurs adresses ou indices. Puis on alloue, avec allocate_variable DS et avec allocate_code_segment CS et on exécute le programme avec run_program, qui interprète les instructions de manière interactive. Enfin, on libère la mémoire allouée.

Sortie attendue:

```
L'insertion de la clé AX dans la table est faite
L'insertion de la clé BX dans la table est faite
L'insertion de la clé CX dans la table est faite
L'insertion de la clé DX dans la table est faite
L'insertion de la clé IP dans la table est faite
L'insertion de la clé ZF dans la table est faite
L'insertion de la clé SF dans la table est faite
L'insertion de la clé x dans la table est faite
Variable 'x' insérée à l'adresse 0, cpt=1
L'insertion de la clé arr dans la table est faite
Variable 'arr' insérée à l'adresse 1, cpt=4
L'insertion de la clé y dans la table est faite
Variable 'y' insérée à l'adresse 5, cpt=1
L'insertion de la clé start dans la table est faite
Label 'start' inséré à l'indice 0
Instruction parsée: mnemonic='MOV', operand1='AX', operand2='x'
L'insertion de la clé loop dans la table est faite
Label 'loop' inséré à l'indice 1
Instruction parsée: mnemonic='ADD', operand1='AX', operand2='y'
Instruction parsée: mnemonic='JMP', operand1='loop', operand2=''
```

```
Débogage de resolve_constants:
Instruction 0
Traitement variable 'x'
search_and_replace: recherche de 'x'
search_and_replace: valeur trouvée=0
search_and_replace: '0' remplacé par '0'
Variable '0' remplacée
Instruction 1
Traitement variable 'y'
search_and_replace: recherche de 'y'
search_and_replace: valeur trouvée=5
search_and_replace: '5' remplacé par '5'
Variable '5' remplacée
Instruction 2
Traitement label 'loop'
search_and_replace: recherche de 'loop'
search_and_replace: valeur trouvée=1
search_and_replace: '1' remplacé par '1'
Label '1' remplacé
L'insertion de la clé DS dans la table est faite
L'insertion de la clé CS dans la table est faite
```

```
L'état initiale de la mémoire :
Le segment de données DS debut=0 et size=6:
memory[0] = 42
memory[1] = 20
memory[2] = 21
memory[3] = 22
memory[4] = 23
memory[5] = 10

L'état initiale des registres de la memoire :
AX = 0
BX = 0
CX= 0
DX = 0
IP = 0
ZF = 0
SF = 0
L'instruction à exécuter est : MOV AX,0
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter
```

```
Chargement depuis DS à l'adresse 0

L'état des registres après l'exécution de l'instruction MOV :
AX = 42
BX = 0
CX = 0
DX = 0
IP = 1
ZF = 0
SF = 0
L'instruction à exécuter est : ADD AX,5
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Chargement depuis DS à l'adresse 5

L'état des registres après l'exécution de l'instruction ADD :
AX = 52
BX = 0
```

```
BX = 0
CX = 0
DX = 0
IP = 2
ZF = 0
SF = 0
L'instruction à exécuter est : JMP 1
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter



L'état des registres après l'exécution de l'instruction JMP :
AX = 52
BX = 0
CX = 0
DX = 0
IP = 1
ZF = 0
SF = 0
L'instruction à exécuter est : ADD AX,5
```

```
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter
s
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Chargement depuis DS à l'adresse 5

L'état des registres après l'exécution de l'instruction ADD :
AX = 82
BX = 0
CX = 0
DX = 0
IP = 2
ZF = 0
SF = 0
L'instruction à exécuter est : JMP 1
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter
q
```

```
L'état finale de la mémoire :
Le segment de données DS debut=0 et size=6:
memory[0] = 42
memory[1] = 20
memory[2] = 21
memory[3] = 22
memory[4] = 23
memory[5] = 10

L'état finale des registres de la memoire :
AX = 82
BX = 0
CX = 0
DX = 0
IP = 2
ZF = 0
SF = 0
exo 1 Libération de la clé start à l'index 46
exo 1 Libération de la clé loop à l'index 58
exo 1 HashMap destroyed: 2 keys freed
exo 1 Libération de la clé arr à l'index 69
```

```
exo 1 Libération de la clé arr à l'index 69
exo 1 Libération de la clé x à l'index 120
exo 1 Libération de la clé y à l'index 121
exo 1 HashMap destroyed: 3 keys freed
exo 1 Libération de la clé AX à l'index 25
exo 1 Libération de la clé BX à l'index 26
exo 1 Libération de la clé CX à l'index 27
exo 1 Libération de la clé DX à l'index 28
exo 1 Libération de la clé IP à l'index 29
exo 1 Libération de la clé SF à l'index 30
exo 1 Libération de la clé ZF à l'index 32
exo 1 HashMap destroyed: 7 keys freed
exo 1 HashMap destroyed: 0 keys freed
Clé non trouvée : SS
exo 1 Libération de la clé CS à l'index 22
exo 1 Libération de la clé DS à l'index 23
exo 1 HashMap destroyed: 2 keys freed
```

Valgrind:

```
==14570==
==14570== HEAP SUMMARY:
==14570==     in use at exit: 0 bytes in 0 blocks
==14570==   total heap usage: 1,695 allocs, 1,695 frees, 174,111 bytes all
ocated
==14570==
==14570== All heap blocks were freed -- no leaks are possible
==14570==
==14570== For lists of detected and suppressed errors, rerun with: -s
==14570== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

main7.c:

L'objectif est de valider les fonctions de l'exercice 7 pour exécuter un programme assembleur de manière interactive.

On alloue un CPU de size 2048 avec cpu_init, on parse le fichier exo7.txt avec parse, générant un ParserResult, ensuite, on appelle resolve_constants pour associer les labels (start, loop) et variables (x, y) à leurs adresses ou indices. Puis on insère les données de data_instructions avec insert_data dans DS et avec allocate_code_segment on alloue et on initialise CS ensuite on exécute le programme avec run_program, qui interprète les instructions de manière interactive. Enfin, on libère la mémoire allouée.

Sortie attendue:

```
L'insertion de la clé SP dans la table est faite
L'insertion de la clé BP dans la table est faite
L'insertion de la clé AX dans la table est faite
L'insertion de la clé BX dans la table est faite
L'insertion de la clé CX dans la table est faite
L'insertion de la clé DX dans la table est faite
L'insertion de la clé IP dans la table est faite
L'insertion de la clé ZF dans la table est faite
L'insertion de la clé SF dans la table est faite
L'insertion de la clé DS dans la table est faite
L'insertion de la clé SS dans la table est faite
L'insertion de la clé x dans la table est faite
Variable 'x' insérée à l'adresse 0, cpt=1
L'insertion de la clé y dans la table est faite
Variable 'y' insérée à l'adresse 1, cpt=1
L'insertion de la clé start dans la table est faite
Label 'start' inséré à l'indice 0
Instruction parsée: mnemonic='MOV', operand1='AX', operand2='x'
Instruction parsée: mnemonic='PUSH', operand1='AX', operand2=''
Instruction parsée: mnemonic='MOV', operand1='AX', operand2='y'
```

```
Instruction parsée: mnemonic='POP', operand1='BX', operand2=''
Instruction parsée: mnemonic='HALT', operand1='', operand2=''
Débogage de resolve_constants:
Instruction 0
Traitement variable 'x'
search_and_replace: recherche de 'x'
search_and_replace: valeur trouvée=0
search_and_replace: '0' remplacé par '0'
Variable '0' remplacée
Instruction 1
Instruction 2
Traitement variable 'y'
search_and_replace: recherche de 'y'
search_and_replace: valeur trouvée=1
search_and_replace: '1' remplacé par '1'
Variable '1' remplacée
Instruction 3
Instruction 4
L'insertion de la clé CS dans la table est faite

L'état initiale de la mémoire :
```

```
Le segment de données DS debut=0 et size=256:
memory[0] = 42
memory[1] = 10
memory[2] = 0
memory[3] = 0
memory[4] = 0
memory[5] = 0
memory[6] = 0
memory[7] = 0
memory[8] = 0
memory[9] = 0
memory[10] = 0
memory[11] = 0
memory[12] = 0
memory[13] = 0
memory[14] = 0
memory[15] = 0
memory[16] = 0
memory[17] = 0
memory[18] = 0
memory[19] = 0
```

```
memory[250] = 0
memory[251] = 0
memory[252] = 0
memory[253] = 0
memory[254] = 0
memory[255] = 0

L'état initiale des registres de la memoire :
AX = 0
BX = 0
CX= 0
DX = 0
IP = 0
ZF = 0
SF = 0
L'instruction à exécuter est : MOV AX,0
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter
.
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter
```

```
Chargement depuis DS à l'adresse 0
Valeur chargée depuis DS[0] = 42

L'état des registres après l'exécution de l'instruction MOV :
AX = 42
BX = 0
CX = 0
DX = 0
IP = 1
ZF = 0
SF = 0
L'instruction à exécuter est : PUSH AX
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Pushing value 42 from AX

L'état des registres après l'exécution de l'instruction PUSH :
AX = 42
BX = 0
```

```
CX = 0
DX = 0
IP = 2
ZF = 0
SF = 0
L'instruction à exécuter est : MOV AX,1
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Chargement depuis DS à l'adresse 1
Valeur chargée depuis DS[1] = 10

L'état des registres après l'exécution de l'instruction MOV :
AX = 10
BX = 0
CX = 0
DX = 0
IP = 3
ZF = 0
SF = 0
```

```
L'instruction à exécuter est : POP BX
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Popping value to BX

L'état des registres après l'exécution de l'instruction POP :
AX = 10
BX = 42
CX = 0
DX = 0
IP = 4
ZF = 0
SF = 0
L'instruction à exécuter est : HALT
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Exécution terminée (HALT)
```

```
L'état des registres après l'exécution de l'instruction HALT :
AX = 10
BX = 42
CX = 0
DX = 0
IP = 5
ZF = 0
SF = 0
IP n'est pas dans les limites
Aucune instruction à exécuter

L'état finale de la mémoire :
Le segment de données DS debut=0 et size=256:
memory[0] = 42
memory[1] = 10
memory[2] = 0
memory[3] = 0
memory[4] = 0
memory[5] = 0
memory[6] = 0
memory[7] = 0
```

```
L'état finale des registres de la memoire :
AX = 10
BX = 42
CX = 0
DX = 0
IP = 5
ZF = 0
SF = 0
exo 1 Libération de la clé start à l'index 46
exo 1 HashMap destroyed: 1 keys freed
exo 1 Libération de la clé x à l'index 120
exo 1 Libération de la clé y à l'index 121
exo 1 HashMap destroyed: 2 keys freed
exo 1 Libération de la clé BP à l'index 18
exo 1 Libération de la clé AX à l'index 25
exo 1 Libération de la clé BX à l'index 26
exo 1 Libération de la clé CX à l'index 27
exo 1 Libération de la clé DX à l'index 28
exo 1 Libération de la clé IP à l'index 29
exo 1 Libération de la clé SF à l'index 30
```

```
exo 1 Libération de la clé ZF à l'index 32
exo 1 Libération de la clé SP à l'index 35
exo 1 HashMap destroyed: 9 keys freed
exo 1 HashMap destroyed: 0 keys freed
exo 1 Libération de la clé CS à l'index 22
exo 1 Libération de la clé DS à l'index 23
exo 1 Libération de la clé SS à l'index 38
exo 1 HashMap destroyed: 3 keys freed
```

Valgrind:

```
==16794==
==16794== HEAP SUMMARY:
==16794==     in use at exit: 0 bytes in 0 blocks
==16794==   total heap usage: 838 allocs, 838 frees, 106,936 bytes allocat
ed
==16794==
==16794== All heap blocks were freed -- no leaks are possible
==16794==
==16794== For lists of detected and suppressed errors, rerun with: -s
==16794== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

main8.c:

L'objectif est de valider les fonctions de l'exercice 8, et surtout l'ajout du segment ES. On veut vérifier que tout fonctionne comme attendu, de la même façon qu'à l'exercice 7, même après l'ajout de ce nouveau segment. On commence par allouer un CPU de taille 2048 grâce à cpu_init puis on parse le fichier exo8.txt grâce à parse, produisant un ParserResult. Ensuite, resolve_constants associe labels et variables à leurs adresses numériques, tandis que les opérandes utilisant ES comme préfixe sont laissées pour être traitées lors de l'exécution. Enfin, insert_data charge la section .DATA dans DS, et allocate_code_segment initialise et alloue CS. Enfin, run_program interprète interactivement les instructions, dont ALLOC et FREE, avant de libérer toute la mémoire.

Sortie attendue:

```
L'insertion de la clé SP dans la table est faite
L'insertion de la clé BP dans la table est faite
L'insertion de la clé AX dans la table est faite
L'insertion de la clé BX dans la table est faite
L'insertion de la clé CX dans la table est faite
L'insertion de la clé DX dans la table est faite
L'insertion de la clé IP dans la table est faite
L'insertion de la clé ZF dans la table est faite
L'insertion de la clé SF dans la table est faite
L'insertion de la clé ES dans la table est faite
L'insertion de la clé DS dans la table est faite
L'insertion de la clé SS dans la table est faite
L'insertion de la clé x dans la table est faite
Variable 'x' insérée à l'adresse 0, cpt=1
L'insertion de la clé y dans la table est faite
Variable 'y' insérée à l'adresse 1, cpt=1
L'insertion de la clé a dans la table est faite
Variable 'a' insérée à l'adresse 2, cpt=1
L'insertion de la clé b dans la table est faite
Variable 'b' insérée à l'adresse 3, cpt=1
L'insertion de la clé start dans la table est faite
Label 'start' inséré à l'indice 0
Instruction parsée: mnemonic='MOV', operand1='AX', operand2='a'
Instruction parsée: mnemonic='MOV', operand1='BX', operand2='b'
Instruction parsée: mnemonic='ALLOC', operand1='', operand2=''
Instruction parsée: mnemonic='MOV', operand1='[ES:AX]', operand2='x'
Instruction parsée: mnemonic='FREE', operand1='', operand2=''
Instruction parsée: mnemonic='HALT', operand1='', operand2=''
Débogage de resolve_constants:
Instruction 0
Traitement variable 'a'
search_and_replace: recherche de 'a'
search_and_replace: valeur trouvée=2
search_and_replace: '2' remplacé par '2'
Variable '2' remplacée par '2'
Instruction 1
Traitement variable 'b'
search_and_replace: recherche de 'b'
search_and_replace: valeur trouvée=3
search_and_replace: '3' remplacé par '3'
Variable '3' remplacée par '3'
Instruction 2
Instruction 3
Traitement variable 'x'
search_and_replace: recherche de 'x'
```

```
search_and_replace: valeur trouvée=0
search_and_replace: '0' remplacé par '0'
Variable '0' remplacée par '0'
Instruction 4
Instruction 5
L'insertion de la clé CS dans la table est faite

L'état initial de la mémoire :
Le segment de données DS debut=0 et size=256:
memory[0] = 3
memory[1] = 10
memory[2] = 50
memory[3] = 3
memory[4] = 0
```

```
memory[255] = 0
Clé non trouvée : ES
Segment de données ES non alloué

L'état initial des registres :
AX = 0
BX = 0
CX = 0
DX = 0
IP = 0
ZF = 0
SF = 0
ES = -1
L'instruction à exécuter est : MOV AX,2
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Destination résolue pour registre AX : 0x4a85be0
Chargement depuis DS à l'adresse 2
Valeur chargée depuis DS[2] = 50

L'état de la mémoire après l'exécution de l'instruction MOV :
AX = 50
BX = 0
CX = 0
DX = 0
IP = 1
ZF = 0
SF = 0
ES = -1
Clé non trouvée : ES
Segment de données ES non alloué
Le segment de données DS debut=0 et size=256:
memory[0] = 3
memory[1] = 10
memory[2] = 50
memory[3] = 3
memory[4] = 0
```

```
memory[255] = 0
L'instruction à exécuter est : MOV BX,3
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Destination résolue pour registre BX : 0x4a85c30
Chargement depuis DS à l'adresse 3
Valeur chargée depuis DS[3] = 3

L'état de la mémoire après l'exécution de l'instruction MOV :
AX = 50
BX = 3
CX = 0
DX = 0
IP = 2
ZF = 0
SF = 0
ES = -1
Clé non trouvée : ES
Segment de données ES non alloué
Le segment de données DS debut=0 et size=256:
memory[0] = 3
memory[1] = 10
memory[2] = 50
memory[3] = 3
memory[4] = 0
```

```
memory[255] = 0
L'instruction à exécuter est : ALLOC
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Allocation du segment ES
Clé non trouvée : ES
Stratégie invalide dans BX (3), utilisation de first fit
L'insertion de la clé ES dans la table est faite
Segment ES alloué : start=646, size=50

L'état de la mémoire après l'exécution de l'instruction ALLOC :
AX = 50
BX = 3
CX = 0
DX = 0
IP = 3
ZF = 0
SF = 0
ES = 646
Le segment de données ES debut=646 et size=50:
memory[646] = 0
memory[647] = 0
```

```
memory[695] = 0
Le segment de données DS debut=0 et size=256:
memory[0] = 3
memory[1] = 10
memory[2] = 50
memory[3] = 3
memory[4] = 0
memory[5] = 0
```

```
memory[255] = 0
L'instruction à exécuter est : MOV [ES:AX],0
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Destination résolue pour [ES:AX] : 0x4a816c0
Chargement depuis DS à l'adresse 0
Valeur chargée depuis DS[0] = 3

L'état de la mémoire après l'exécution de l'instruction MOV :
AX = 50
BX = 3
CX = 0
DX = 0
IP = 4
ZF = 0
SF = 0
ES = 646
Le segment de données ES debut=646 et size=50:
memory[646] = 0
memory[647] = 0
```

```
memory[695] = 0
Le segment de données DS debut=0 et size=256:
memory[0] = 3
memory[1] = 10
memory[2] = 50
memory[3] = 3
memory[4] = 0
```

```
memory[255] = 0
L'instruction à exécuter est : FREE
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Libération du segment ES
La suppression de la clé ES de la table est faite

L'état de la mémoire après l'exécution de l'instruction FREE :
AX = 50
BX = 3
CX = 0
DX = 0
IP = 5
ZF = 0
SF = 0
ES = -1
Clé non trouvée : ES
Segment de données ES non alloué
Le segment de données DS debut=0 et size=256:
memory[0] = 3
memory[1] = 10
memory[2] = 50
memory[3] = 3
memory[4] = 0
```

```
memory[254] = 0
memory[255] = 0
L'instruction à exécuter est : HALT
Si vous voulez exécuter cette instruction veuillez appuyer sur entrer(
), ou sur 'q' pour quitter


Programme terminé avec HALT

L'état final de la mémoire :
Le segment de données DS debut=0 et size=256:
memory[0] = 3
memory[1] = 10
memory[2] = 50
memory[3] = 3
memory[4] = 0
memory[5] = 0
```

```
memory[254] = 0
memory[255] = 0
Segment de données ES non alloué

L'état final des registres :
AX = 50
BX = 3
CX = 0
DX = 0
IP = 6
ZF = 0
SF = 0
ES = -1
exo 1 Libération de la clé start à l'index 46
exo 1 HashMap destroyed: 1 keys freed
exo 1 Libération de la clé a à l'index 97
exo 1 Libération de la clé b à l'index 98
exo 1 Libération de la clé x à l'index 120
exo 1 Libération de la clé y à l'index 121
exo 1 HashMap destroyed: 4 keys freed
exo 1 Libération de la clé BP à l'index 18
exo 1 Libération de la clé ES à l'index 24
exo 1 Libération de la clé AX à l'index 25
exo 1 Libération de la clé BX à l'index 26
exo 1 Libération de la clé CX à l'index 27
exo 1 Libération de la clé DX à l'index 28
exo 1 Libération de la clé IP à l'index 29
exo 1 Libération de la clé SF à l'index 30
exo 1 Libération de la clé ZF à l'index 32
exo 1 Libération de la clé SP à l'index 35
exo 1 HashMap destroyed: 10 keys freed
exo 1 HashMap destroyed: 0 keys freed
exo 1 Libération de la clé CS à l'index 22
exo 1 Libération de la clé DS à l'index 23
exo 1 Libération de la clé SS à l'index 38
exo 1 HashMap destroyed: 3 keys freed
```

Valgrind:

```
==37494==
==37494== HEAP SUMMARY:
==37494==     in use at exit: 0 bytes in 0 blocks
==37494==   total heap usage: 1,230 allocs, 1,230 frees, 163,630 bytes allocated
==37494==
==37494== All heap blocks were freed -- no leaks are possible
==37494==
==37494== For lists of detected and suppressed errors, rerun with: -s
==37494== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Analyse des performances:

L’évaluation du simulateur a montré une exécution fluide et réussie : le parsing détecte et insère correctement les variables (x, y, a, b) dans la constant_pool, les segments mémoire (DS, SS, CS) sont alloués dynamiquement selon les besoins et libérés sans fuite, et les instructions (MOV, ADD, PUSH, POP, JMP, ALLOC, FREE) produisent les résultats attendus sur les registres. Les tests Valgrind confirment une bonne gestion de la mémoire (par exemple, 535 allocations et 535 libérations pour exo7.txt), attestant d’un taux de libération de 100 %.

Bien que limité aux instructions de base et avec une détection d’erreurs qui pourrait être améliorée (dépassements de pile, accès hors segments), le

simulateur bénéficie d'une implémentation optimisée grâce aux tables de hachage en temps constant.
À l'avenir, l'ajout de nouveaux mnémoniques, un renforcement des contrôles des éventuelles erreurs et une interface graphique pourraient être de bonnes améliorations à apporter.

Conclusion:

Le projet de simulateur de CPU a bien fonctionné, avec une implémentation réussie des huit exercices. La table de hachage a assuré une gestion efficace des données dès le départ, et les étapes suivantes ont ajouté le parsing des instructions et la gestion des segments mémoire (DS, SS, CS, ES). Tous les tests ont réussi, et Valgrind a confirmé l'absence de fuites mémoire dans les tests effectués pour chaque exercice. Nous avons rencontré quelques défis, comme les erreurs d'allocation, mais que nous avons fini par réussir à résoudre.
Malgré ça, il reste une piste d'amélioration: le parsing rencontre un problème dans l'exercice 8 avec des valeurs numériques directes (par exemple: MOV AX, 5). La gestion des erreurs pourrait aussi être plus dure, surtout pour les stack overflow.
Pour l'améliorer, on pourrait ajouter des instructions, corriger l'erreur de l'exercice 8 en gérant mieux les constantes, et renforcer la détection d'erreurs. Ce projet a été une occasion d'apprendre à mieux gérer la gestion mémoire et à mettre en pratique toutes les structures de données étudiées.

Annexes:

- **cpu_init** (exo7.c) : Initialise le CPU avec les registres et les segments DS et SS:

```
////////////////////////////////////////////////////////////////////
CPU* cpu_init(int memory_size) {
    CPU* cpu = (CPU*)malloc(sizeof(CPU));
    if (!cpu) {
        printf("Erreur dans l'allocation du cpu\n");
        return NULL;
    }

    cpu->memory_handler = memory_init(memory_size);
    if (!cpu->memory_handler) {
        printf("Erreur dans l'allocation du memory Handler\n");
        free(cpu);
        return NULL;
    }

    cpu->constant_pool = hashmap_create();
    if (!cpu->constant_pool) {
        printf("Erreur dans l'allocation de la HashMap\n");
        free_memory_handler(cpu->memory_handler);
        free(cpu);
        return NULL;
    }

    cpu->context = hashmap_create();
    if (!cpu->context) {
        printf("Erreur dans l'allocation de la HashMap\n");
        hashmap_destroy(cpu->constant_pool);
        free_memory_handler(cpu->memory_handler);
        free(cpu);
        return NULL;
    }

    // Allouer les registres
    int *sp = (int *)malloc(sizeof(int));
    int *bp = (int *)malloc(sizeof(int));
    int *ax = (int *)malloc(sizeof(int));
```

```c
    int *bx = (int *)malloc(sizeof(int));
    int *cx = (int *)malloc(sizeof(int));
    int *dx = (int *)malloc(sizeof(int));
    int *ip = (int *)malloc(sizeof(int));
    int *zf = (int *)malloc(sizeof(int));
    int *sf = (int *)malloc(sizeof(int));

    if (!sp || !bp || !ax || !bx || !cx || !dx || !ip || !zf || !sf) {
        free(sp); free(bp); free(ax); free(bx); free(cx); free(dx); free(ip);
        hashmap_destroy(cpu->context);
        hashmap_destroy(cpu->constant_pool);
        free_memory_handler(cpu->memory_handler);
        free(cpu);
        return NULL;
    }

    // Initialiser les valeurs
    *sp = 0; *bp = 0; *ax = 0; *bx = 0; *cx = 0; *dx = 0; *ip = 0; *zf = 0; *s

    // Insérer les registres dans context
    if (hashmap_insert(cpu->context, "SP", sp) != 0 ||
        hashmap_insert(cpu->context, "BP", bp) != 0 ||
        hashmap_insert(cpu->context, "AX", ax) != 0 ||
        hashmap_insert(cpu->context, "BX", bx) != 0 ||
        hashmap_insert(cpu->context, "CX", cx) != 0 ||
        hashmap_insert(cpu->context, "DX", dx) != 0 ||
        hashmap_insert(cpu->context, "IP", ip) != 0 ||
        hashmap_insert(cpu->context, "ZF", zf) != 0 ||
        hashmap_insert(cpu->context, "SF", sf) != 0) {
        printf("Erreur dans l'insertion des registres dans la HashMap\n");
        free(sp); free(bp); free(ax); free(bx); free(cx); free(dx); free(ip);
        hashmap_destroy(cpu->context);
        hashmap_destroy(cpu->constant_pool);
        free_memory_handler(cpu->memory_handler);
        free(cpu);
        return NULL;
    }
```

```c
    // Créer le segment de données DS (taille 256)
    if (create_segment(cpu->memory_handler, "DS", 0, 256) != 0) {
        printf("Impossible d'allouer le segment DS\n");
        hashmap_destroy(cpu->context);
        hashmap_destroy(cpu->constant_pool);
        free_memory_handler(cpu->memory_handler);
        free(cpu);
        return NULL;
    }
    // Créer le segment de pile SS (taille 128, après DS)
    Segment* ds = (Segment*)hashmap_get(cpu->memory_handler->allocated, "DS");
    int ss_start = ds->start + ds->size + 256; // Réserve un espace pour DS
    int ss_size = 128;
    if (ss_start + ss_size > memory_size) {
        printf("Erreur: mémoire insuffisante pour SS (ss_start=%d, ss_size=%d,
               ss_start, ss_size, memory_size);
        hashmap_destroy(cpu->context);
        hashmap_destroy(cpu->constant_pool);
        free_memory_handler(cpu->memory_handler);
        free(cpu);
        return NULL;
    }
    if (create_segment(cpu->memory_handler, "SS", ss_start, ss_size) != 0) {
        printf("Impossible d'allouer le segment SS\n");
        hashmap_destroy(cpu->context);
        hashmap_destroy(cpu->constant_pool);
        free_memory_handler(cpu->memory_handler);
        free(cpu);
        return NULL;
    }

    // Initialiser BP et SP
    *bp = ss_start + ss_size - 1; // Bas de la pile
    *sp = ss_start + ss_size;     // Haut de la pile (décrémente lors de PUSH)

    return cpu;
```

- **create_segment** (exo2.c) : Alloue un segment mémoire:

```c
int create_segment(MemoryHandler* handler, const char* name, int start, int size){

    if(!handler || !name){
        printf("MemoryHandler ou nom de segment invalide\n");
        return 1;
    }

    if(start < 0 || start + size > handler->total_size){
        printf("Adresse ou taille invalide\n");
        return 1;
    }

    // Vérifier s'il y a un espace mémoire libre suffisant
    Segment* prev;
    Segment* seg_libre = find_free_segment(handler, start, size, &prev);
    if(!seg_libre){
        printf("Aucun espace mémoire libre suffisant est disponible à l'adresse indiquée\n");
        return 1;
    }

    // Créer un nouveau segment
    Segment* new_seg = segment_init(size);
    if(!new_seg){
        return 1;
    }
    new_seg->start = start;  // Ajuster start ici car segment_init ne le prend pas

    // Ajouter à la table de hachage
    int res = hashmap_insert(handler->allocated, name, new_seg);
    if(res == 1){
        free(new_seg);  // Correction : libérer si l'insertion échoue
        return 1;
    }

    // Stocker dans memory, une boucle for car le nb de case occupé par un segment depend de sa taille
    for(int i = start; i < start + size; i++){
```

```c
        handler->memory[i] = new_seg;
    }

    // Ajuster la liste des segments libres
    if(seg_libre->start == start && seg_libre->size == size){
        // Si le segment libre est entièrement consommé, on le supprime
        if(prev){
            prev->next = seg_libre->next;
        }else{
            handler->free_list = seg_libre->next;
        }
        free(seg_libre);
    }else{
        // Simplifié : on ajuste seulement la partie après, sans créer de segment avant
        int original_end = seg_libre->start + seg_libre->size;
        seg_libre->start = start + size;
        seg_libre->size = original_end - seg_libre->start;  // Correction : calculer la taille restante
    }
    return 0;
}
```

- **execute_instruction** (exo7.c) : Exécute une instruction après résolution des opérandes:

```c
}

////////////////////////////////////////////////////////////////////
int execute_instruction(CPU* cpu, Instruction* instr) {
    if (!cpu || !instr || !instr->mnemonic) {
        printf("Un des paramètres de la fonction execute_instruction est nul\n");
        return 0;
    }
    char* mnemonic = instr->mnemonic;
    void* src = NULL;
    void* dest = NULL;

    if (strcmp(mnemonic, "MOV") == 0) {
        if (!instr->operand1 || !instr->operand2) {
            printf("Un des opérandes est null pour MOV\n");
            return 0;
        }
        dest = register_addressing(cpu, instr->operand1);
        if (!dest) {
            printf("operand1 '%s' n'est pas un registre valide\n", instr->operand1);
            return 0;
        }
        char* pattern = "^[0-9]+$";
        if (matches(pattern, instr->operand2)) {
            int addr = atoi(instr->operand2);
            printf("Chargement depuis DS à l'adresse %d\n", addr);
            src = load(cpu->memory_handler, "DS", addr);
            if (!src) {
                printf("Impossible de charger la valeur à l'adresse %d dans DS (src=NULL)\n", addr);
                return 0;
            }
            printf("Valeur chargée depuis DS[%d] = %d\n", addr, *(int*)src);
        } else {
            src = resolve_addressing(cpu, instr->operand2);
            if (!src) {
                printf("operand2 '%s' n'est pas valide\n", instr->operand2);
                return 0;
```

```c
        }
        return handle_instruction(cpu, instr, src, dest);
    }

    if (strcmp(mnemonic, "PUSH") == 0) {
        if (strcmp(instr->operand2, "") != 0) {
            printf("Erreur : PUSH accepte un seul opérande\n");
            return 0;
        }
        src = register_addressing(cpu, instr->operand1);
        if (!src) {
            printf("operand1 '%s' n'est pas un registre valide pour PUSH\n", instr->operand1);
            return 0;
        }
        printf("Pushing value %d from %s\n", *(int*)src, instr->operand1);
        return handle_instruction(cpu, instr, src, NULL);
    }

    if (strcmp(mnemonic, "POP") == 0) {
        if (strcmp(instr->operand2, "") != 0) {
            printf("Erreur : POP accepte un seul opérande\n");
            return 0;
        }
        dest = register_addressing(cpu, instr->operand1);
        if (!dest) {
            printf("operand1 '%s' n'est pas un registre valide pour POP\n", instr->operand1);
            return 0;
        }
        printf("Popping value to %s\n", instr->operand1);
        return handle_instruction(cpu, instr, NULL, dest);
    }

    if (strcmp(mnemonic, "HALT") == 0) {
        if(strcmp(instr->operand2, "") != 0 || strcmp(instr->operand1, "") != 0) {
            printf("Erreur : HALT n'accepte aucun opérande\n");
            return 0;
```

```c
        }
        return handle_instruction(cpu, instr, NULL, NULL);
    }

    printf("Instruction '%s' non reconnue\n", mnemonic);
    return 0;
}
```

- **Sortie Valgrind de l'exercice 8 :**

==37494==
==37494== HEAP SUMMARY:
==37494== in use at exit: 0 bytes in 0 blocks
==37494== total heap usage: 1,230 allocs, 1,230 frees, 163,630 bytes allocated
==37494==
==37494== All heap blocks were freed -- no leaks are possible
==37494==
==37494== For lists of detected and suppressed errors, rerun with: -s
==37494== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

Références:

Pour réussir à mener à bien notre projet, ainsi que pour nous assurer de la justesse et de l'efficacité de notre code, nous nous sommes appuyés sur plusieurs ressources clés. La **documentation C**, notamment les pages de manuel (man pages) des fonctions standard telles que malloc, free et strdup, a été consultée pour assurer une utilisation correcte de ces fonctions essentielles à la gestion dynamique de la mémoire, mais aussi pour rechercher des fonctions pouvant nous être utiles lorsque nous étions en difficulté. Le **Valgrind User Manual** a joué un rôle important dans la maîtrise de l'outil de débogage mémoire, nous permettant d'interpréter les résultats pour détecter et éventuellement corriger les fuites mémoires et autres erreurs. Enfin, l'**assistance de Grok** (xAI) a été utilisée pour résoudre des problèmes spécifiques rencontrés lors des phases d'implémentation et de débogage.